AI시대, 차세대 클라우드의 새로운 비전과 기술

openKcloud DevDay 2025

AI반도체의 현재와 미래

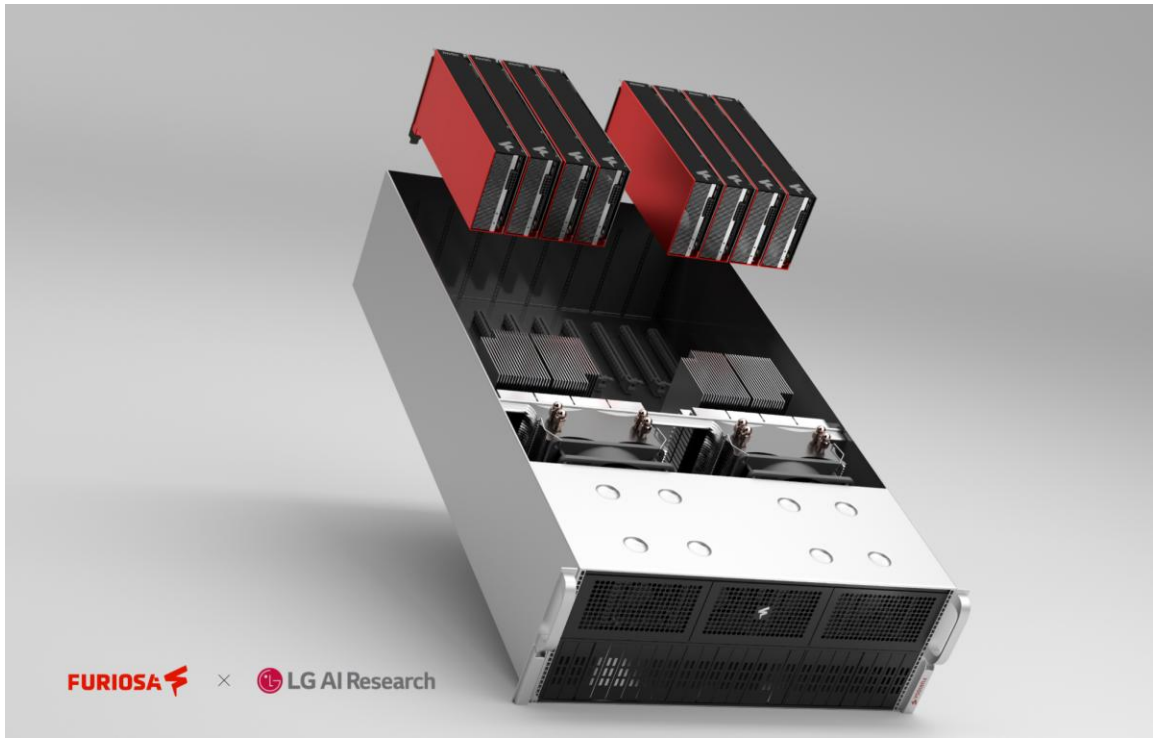
FuriosaAI는 어디로 달려가고 있는가

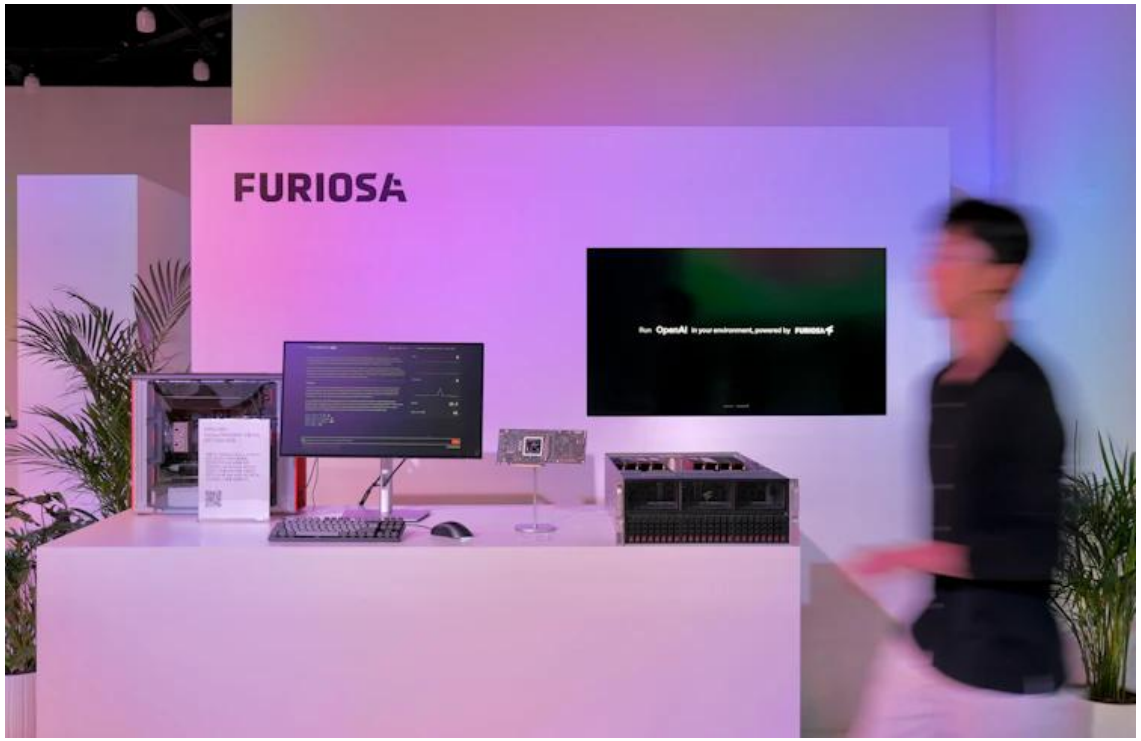
퓨리오사에이아이
이병찬

목 차

1. AI Accelerators in the World
2. FuriosaAI RNGD
3. furiosa-llm: Serving Framework
4. furiosa-torch: PyTorch Integration
5. tcp-ir: Low-Level Programming Interface

FuriosaAI in 2025





1. AI Accelerators in the World

AI Accelerators in the World

NVIDIA

AMD

AI Accelerators in the World

groq®

cerebras

sambanova

GRAPHCORE

tenstorrent

OPENCLOUD

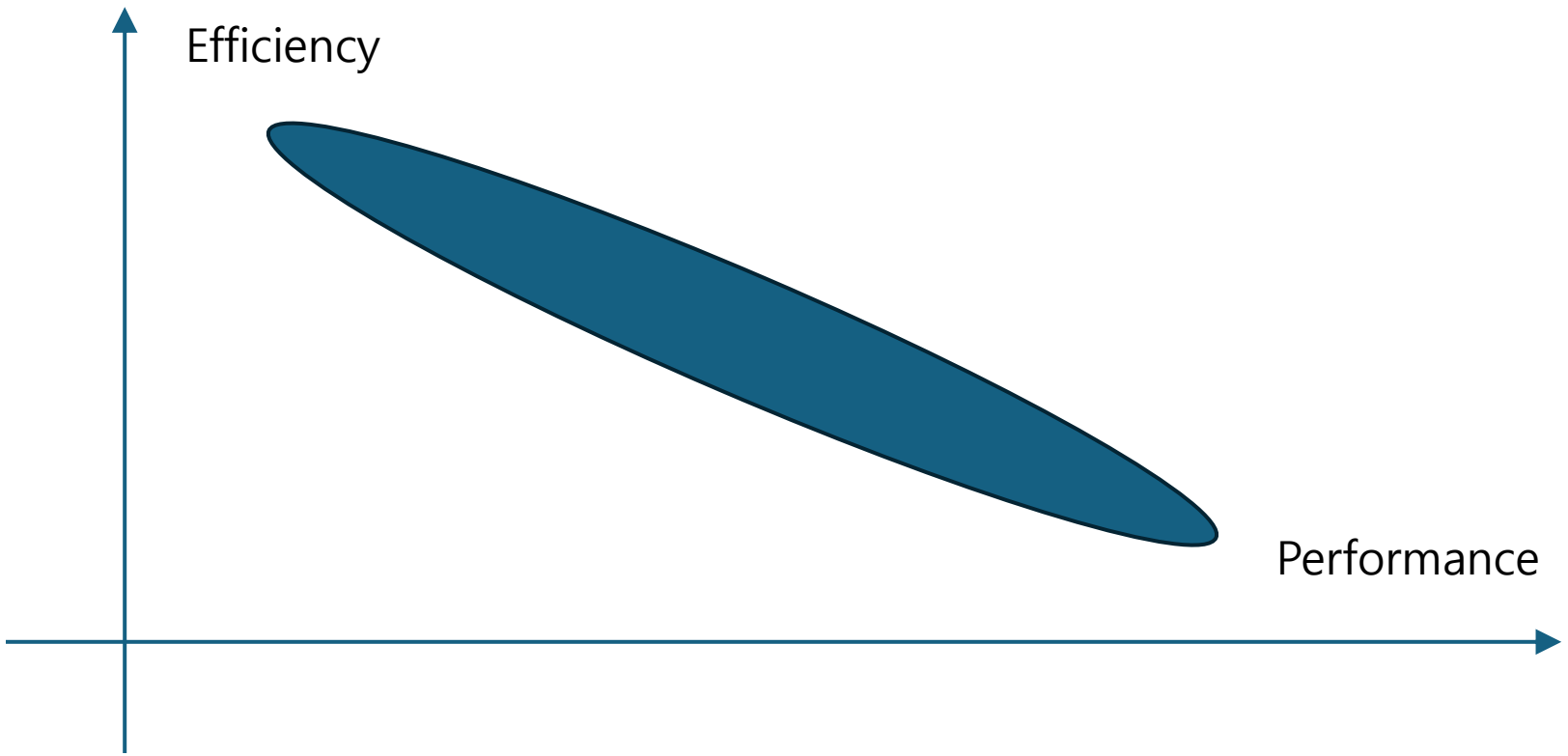
AI Accelerators in the World

FURIOSA

rebellions_

DEEPX

AI Accelerators in the World



Question (Doubt)

NVIDIA의 대안을 왜 고려해야 하나요?

OPENCLOUD

Question (Doubt)

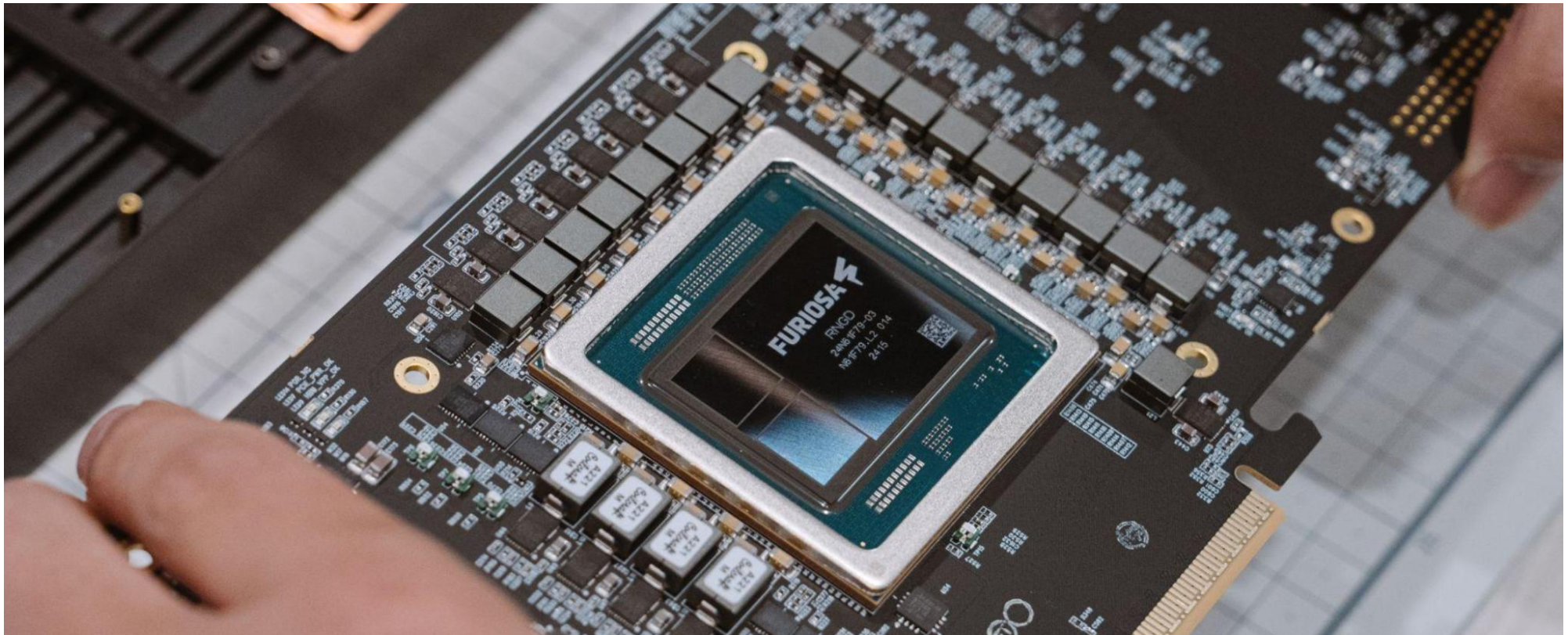
Our mission is to build AI chips capable of running the world's most advanced models efficiently.

We believe this is a critical lever in making AI computing more sustainable for the next generation.

2. FuriosaAI RNGD

FuriosaAI RNGD

- TCP: A Tensor Contraction Processor for AI Workloads
  - Presented at ISCA: International Symposium on Computer Architecture, 2024

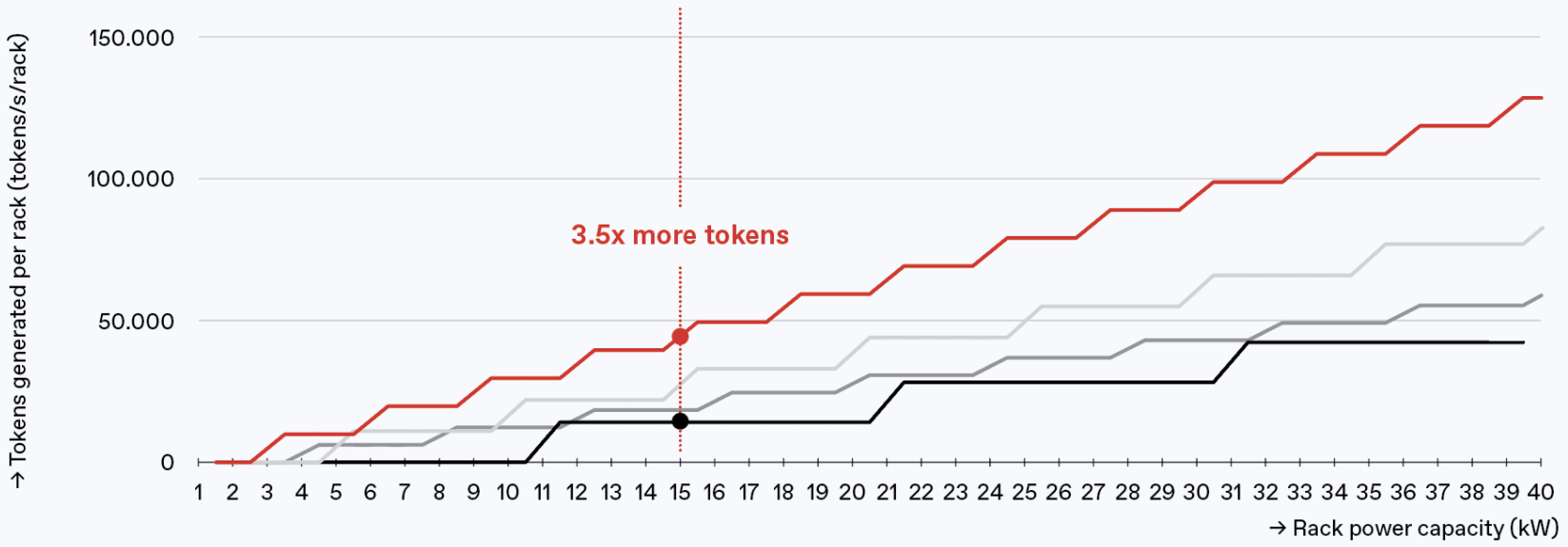


FuriosaAI RNGD

3.5x more compute-per-rack than GPUs

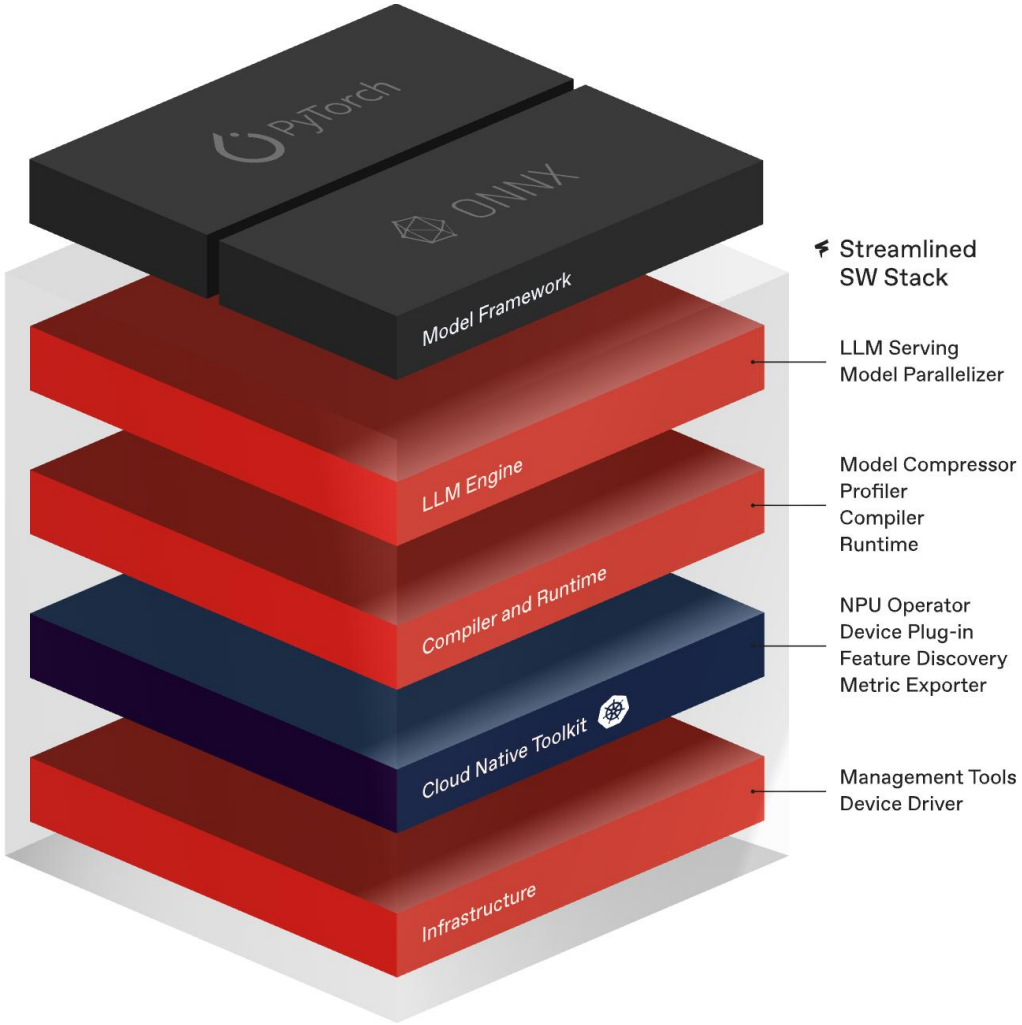
Rack-level Performance for RNGD vs. GPUs
with LLama 3.1 8B FP8

5x RNGD Servers	3x H100 PCIe System	3x L40s System	1x H100 SXM System
49.412 tokens/s	32.957 tokens/s	18.425 tokens/s	14.095 tokens/s



Comparison of rack-level token generation performance under a fixed 15 kW power budget, with all systems running identical workloads

FuriosaAI RNGD



Question (Doubt)

NVIDIA를 대체하는게 가능(하긴)한가요?

3. furiosa-llm: Serving Framework

furiosa-llm: Serving Framework

- OpenAI-compatible API Server
- vLLM compatible interface

```
vllm serve LGAI-EXAONE/EXAONE-4.0-32B \
  --enable-auto-tool-choice --tool-call-parser hermes --reasoning-parser deepseek_r1
```

```
furiosa-llm serve LGAI-EXAONE/EXAONE-4.0-32B \
  --enable-auto-tool-choice --tool-call-parser hermes --reasoning-parser deepseek_r1
```

furiosa-llm: Serving Framework

- OpenAI-compatible API Server
- vLLM compatible interface

```
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [
      {
        "role": "developer",
        "content": "You are a helpful assistant."
      },
      {
        "role": "user",
        "content": "Hello!"
      }
    ]
  }'
```

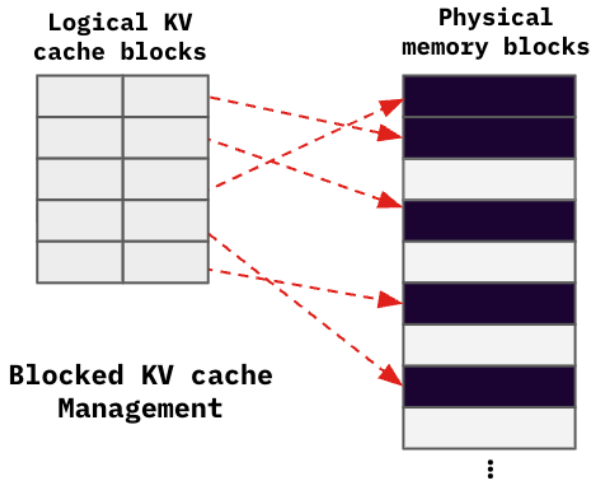
furiosa-llm: Serving Framework

- OpenAI-compatible API Server
- vLLM compatible interface

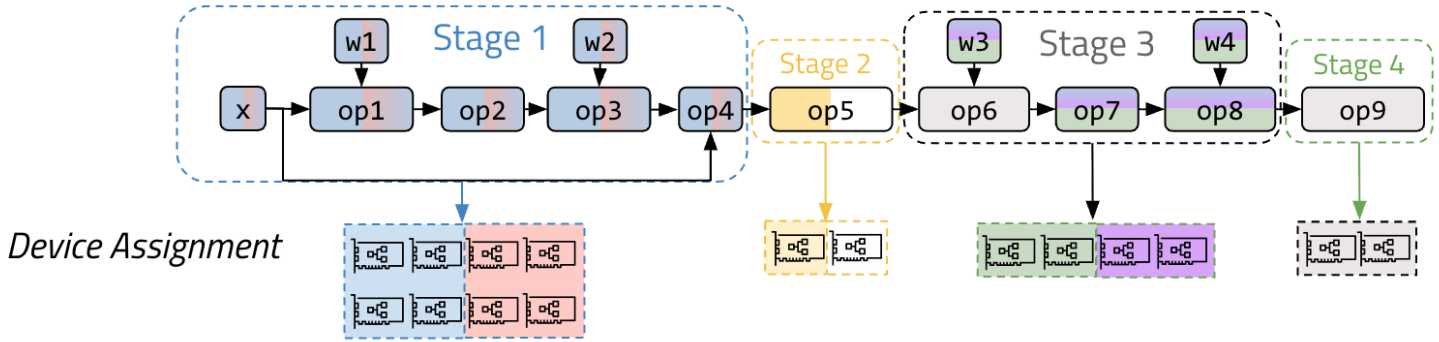
```
client = AsyncOpenAI(base_url=base_url)

stream_chat_completion =
  await client.chat.completions.create(
    messages=[{
      "role": "user",
      "content": "What is the capital of France?"
    }],
    stream=True,
  )

async for chunk in stream_chat_completion:
  print(
    chunk.choices[0].delta.content or "",
    end="",
    flush=True
  )
```

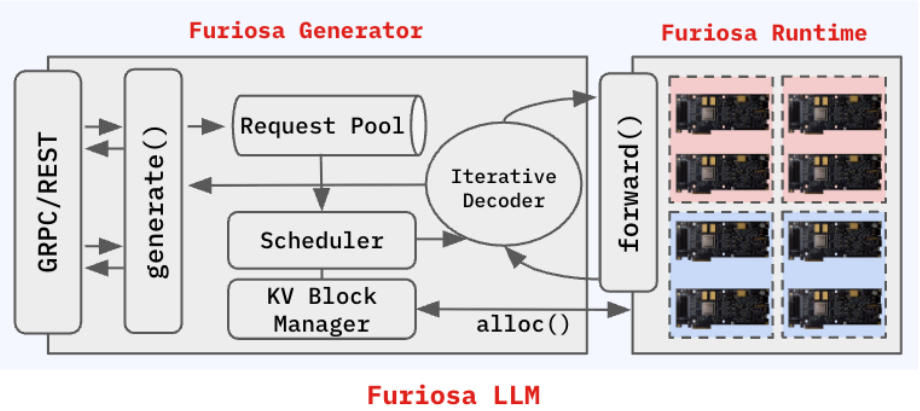
furiosa-llm: Serving Framework

- High throughput serving with SOTA optimization
  - Parallelism
  - Paged Attention
  - Continuous Batching
  - Prefix Caching
  - Sarathi Serve



Pipeline over data/tensor-parallelized stages



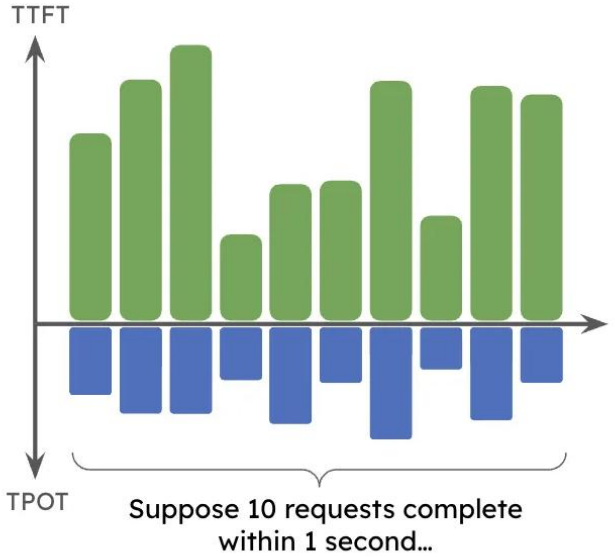


furiosa-llm: Serving Framework

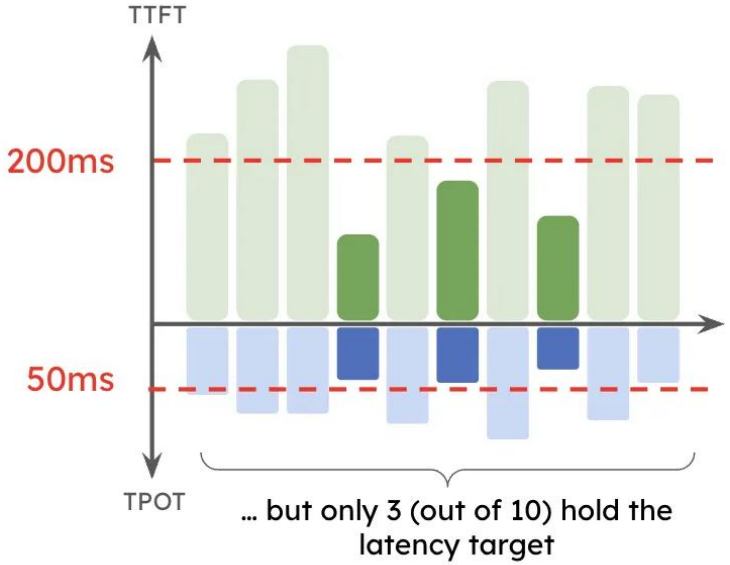
- Go to High Goodput

High Throughput ≠ High Goodput

Throughput = completed request / time
= **10** req / s



Goodput = completed requests **within SLO** / time
= **3** req / s



furiosa-llm: Serving Framework

- Supported Models
  - Llama 3.1/3.3 models
  - Qwen 2/2.5/3.0 models
  - Exaone 3.5/4.0 models
  - Upstage SOLAR models

Pre-built models are available here:

https://huggingface.co/furiosa-ai

furiosa-ai/DeepSeek-R1-Distill-Llama-8B	BF16
furiosa-ai/DeepSeek-R1-Distill-Llama-70B	BF16
furiosa-ai/DeepSeek-R1-Distill-Qwen-7B	BF16
furiosa-ai/DeepSeek-R1-Distill-Qwen-14B	BF16
furiosa-ai/DeepSeek-R1-Distill-Qwen-32B	BF16
furiosa-ai/EXAONE-3.5-7.8B-Instruct	BF16
furiosa-ai/EXAONE-3.5-32B-Instruct	BF16
furiosa-ai/Llama-3.1-8B-Instruct	BF16
furiosa-ai/Llama-3.1-8B-Instruct-FP8	FP8 quantized
furiosa-ai/Llama-3.3-70B-Instruct	BF16
furiosa-ai/Llama-3.3-70B-Instruct-INT8	INT8 weight quantization
furiosa-ai/Qwen2.5-7B-Instruct	BF16
furiosa-ai/Qwen2.5-14B-Instruct	BF16
furiosa-ai/Qwen2.5-32B-Instruct	BF16
furiosa-ai/Qwen2.5-Coder-7B-Instruct	BF16
furiosa-ai/Qwen2.5-Coder-14B-Instruct	BF16
furiosa-ai/Qwen2.5-Coder-32B-Instruct	BF16

furiosa-llm: Serving Framework

- Kubernetes Integration
  - Feature Discovery
  - Metrics Exporter
  - Device Plugin
  - NPU Operator
  - Dynamic Resource Allocation(DRA)

furiosa-llm: Serving Framework

- distributed LLM inference framework
  - llm-d (based on Gateway API Inference Extension)
    - Prefix-aware routing
    - Inference-workload routing
  - (TODO) Ray Serve

4. furiosa-torch: PyTorch Integration

furiosa-torch: PyTorch Integration

- Torch with CUDA

```
device = torch.device("cuda")

a = torch.randn(1024, 1024, device=device)
b = torch.randn(1024, 1024, device=device)

c = torch.matmul(a, b)
```

furiosa-torch: PyTorch Integration

- Torch with CUDA
- Torch with RNGD

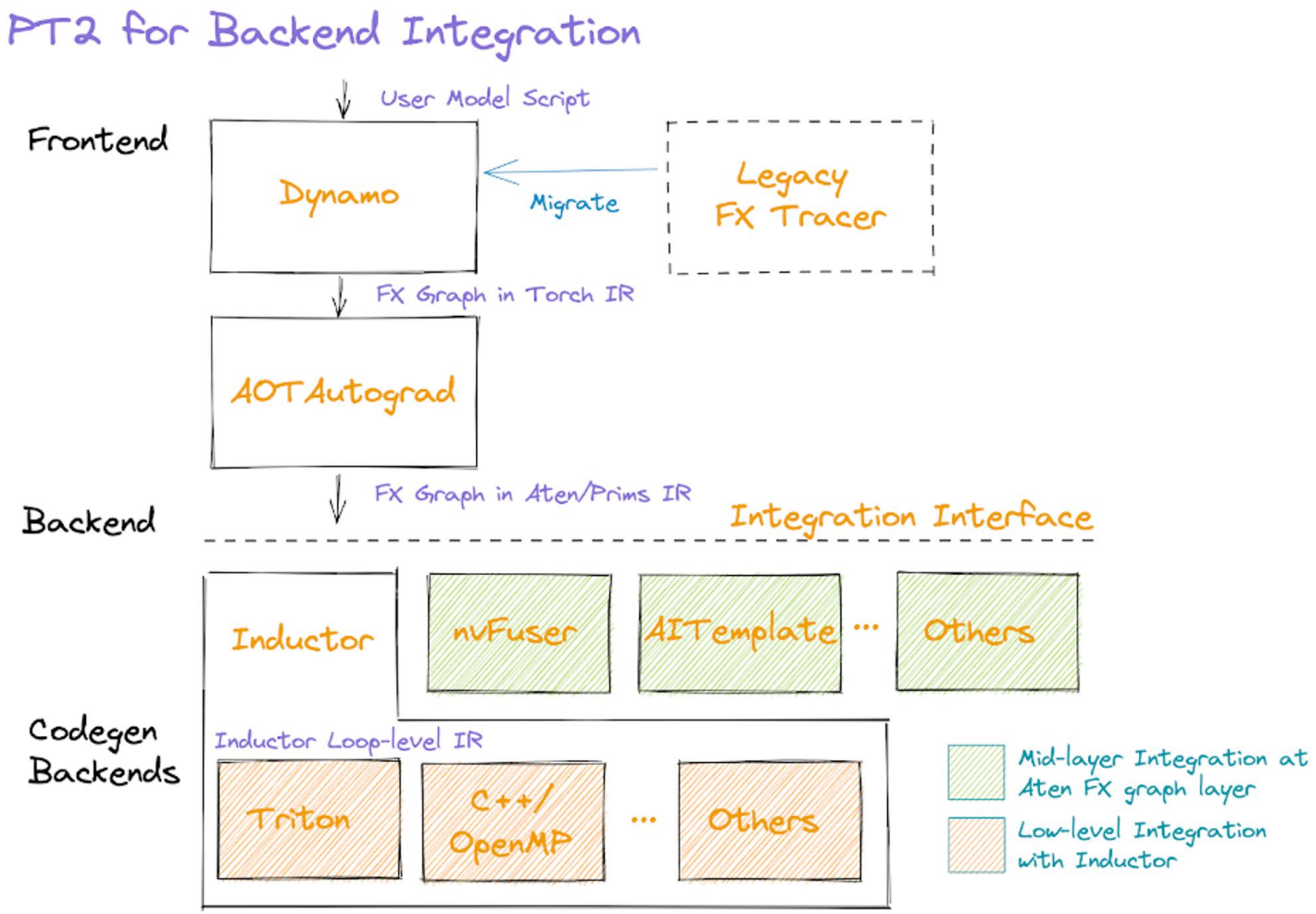
```
device = torch.device("rngd:0")

a = torch.rand([32, 2, 256], dtype=torch.bfloat16)
b = torch.rand([32, 2, 256], dtype=torch.bfloat16)

c = a.to(device) + b.to(device)
assert torch.all(c.cpu() == a + b)
```

furiosa-torch: PyTorch Integration

- Torch with CUDA
- Torch with RNGD
- Torch Compile with RNGD

```
def model(a, b):
    return (a - b) * (a + b)


model = torch.compile(
    model, backend=furiosa.torch.backend.backend)

device = torch.device("rngd:0")

a = torch.rand([32, 2, 256], dtype=torch.bfloat16)
b = torch.rand([32, 2, 256], dtype=torch.bfloat16)
z = model(a.to(device), b.to(device))

assert torch.allclose(
  z.cpu(),
  (a - b) * (a + b),
  atol=0.1, rtol=0.1,
)
```

furiosa-torch: PyTorch Integration



nn.Module

↓

fx.GraphModule

↓

AtenIR

↓

FuriosaIR (High-Level)

↓

FuriosaIR (Low-Level)

furiosa-torch: PyTorch Integration

- Compiling nn.Module

```
class M(torch.nn.Module):
    def forward(self, x, y):
        return (x + y) + y

device = torch.device("rngd:0")
m = M()
compiled_m = torch.compile(m, backend=backend.backend)

inputs = (
    torch.rand([32, 2, 256], dtype=torch.bfloat16),
    torch.rand([32, 2, 256], dtype=torch.bfloat16),
)

answer = m(*inputs)

inputs = [i.to(device=device) for i in inputs]
output = compiled_m(*inputs)
```

furiosa-torch: PyTorch Integration

- Compiling nn.Module
- Compiling fx.GraphModule

```
# Build a GraphModule for (a + b) + b
g = torch.fx.Graph()
i0 = g.placeholder("i0")
i1 = g.placeholder("i1")
o0 = g.call_function(torch.ops.aten.add, (i0, i1))
o1 = g.call_function(torch.ops.aten.add, (o0, i1))
g.output(o1)
gm = torch.fx.GraphModule(torch.nn.Module(), g)

a = torch.rand([32, 2, 256], dtype=torch.bfloat16)
b = torch.rand([32, 2, 256], dtype=torch.bfloat16)

inputs = [a.to(device), b.to(device)]
outputs = CompileModule(gm, inputs)(*inputs)
```

furiosa-torch: PyTorch Integration

- Compiling nn.Module
- Compiling fx.GraphModule
- Example: resnet152

```
model = torchvision.models.resnet152(weights=
  torchvision.models.ResNet152_Weights.IMAGENET1K_V1)
model.eval()

model = torch.compile(
  model=model,
  backend=furiosa.torch.backend.backend,
)
```

5. tcp-ir: Low-Level Programming Interface

tcp-ir: Low-Level Programming Interface

- NVIDIA
  - CUDA
  - CUTLASS
- FuriosaAI?

tcp-ir: Low-Level Programming Interface

- Tensor Contraction Processor!
- User scenario
  - Finance
  - Quantum
    - CUDA-Q?

tcp-ir: Low-Level Programming Interface

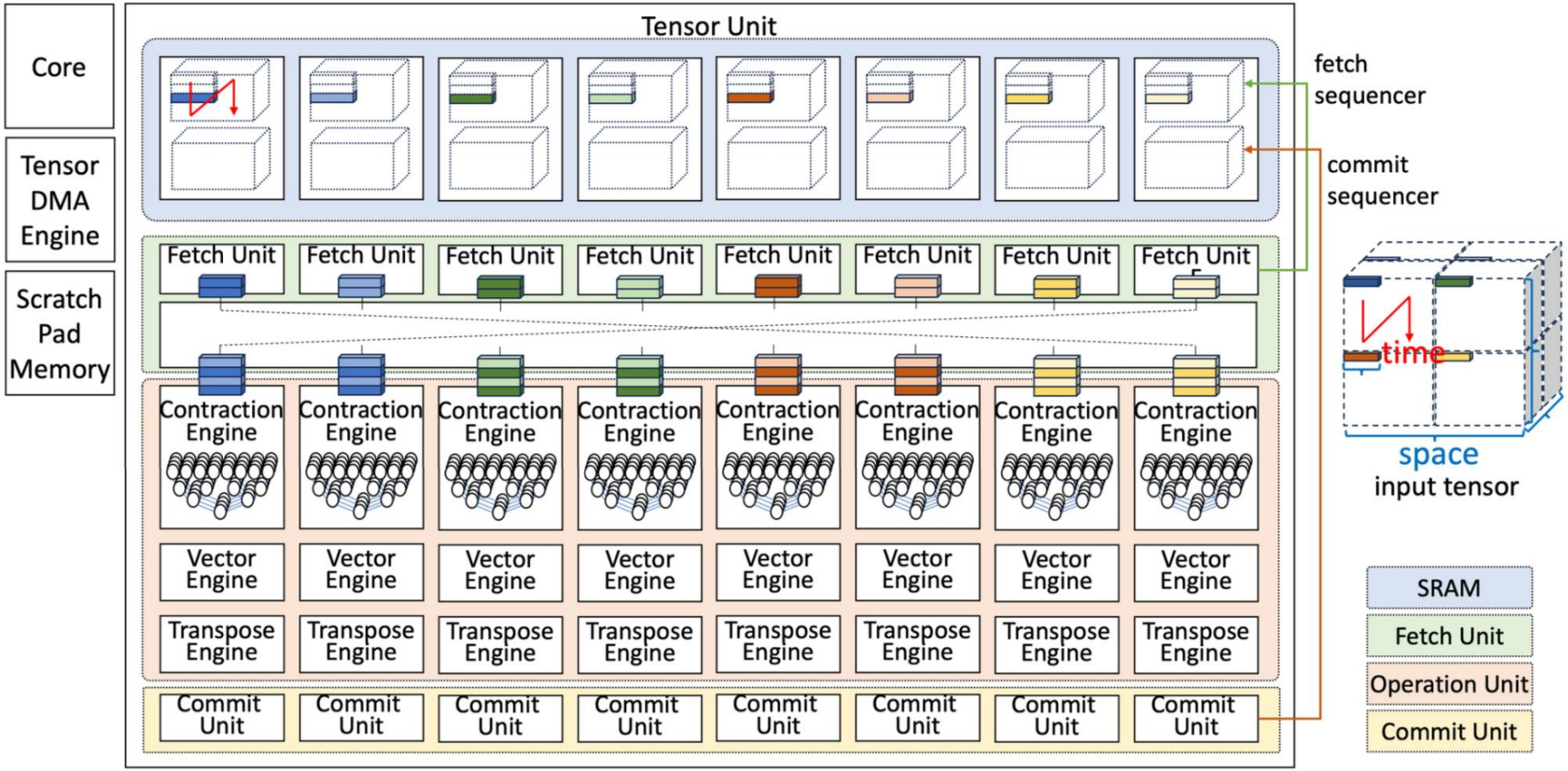


Fig. 3: A processing element of TCP with eight slices.

tcp-ir: Low-Level Programming Interface

- DMA

```
dram = torch.rand([64, 16, 256]).to(device)
sram = empty_sram([64, 16, 256], device=device)
rngd = native_torch.get_device(device.index)
asyncio.run(
    rngd.execute(
        hal.execute(
            hal.pert.copy_local(
                dram,
                sram,
                ir.Sequencer.from_tensor(dram),
                ir.Sequencer.from_tensor(sram),
            )
        )
    )
)
```

In progress: PoC-level code

tcp-ir: Low-Level Programming Interface

- DMA
- Fetch Engine

```
origin = torch.rand([1024, 256], dtype=torch.bfloat16)
x = tensor_to_sram(origin.view([64, 32, 128]).to(device=device))
input_shape = ir.LoweredShape.from_tensor(x)
output_shape = ir.LoweredShape.from_lowered_axis(
    [input_shape.partitioning[0]],
    [
        (input_shape.in_slice[0].label, 1, 8, 128 * 4),
        (input_shape.in_slice[0].label, 8, 4, 128),
        (input_shape.in_slice[1].label, 1, 128, 1),
    ],
)

fetch_engine = lla.FetchEngine(
    input_shape=input_shape,
    input_dtype=x.dtype,
    output_shape=output_shape,
    output_dtype=None,
)
```

In progress: PoC-level code

tcp-ir: Low-Level Programming Interface

- DMA
- Fetch Engine
- Contraction Engine

```
origin_input = torch.randn([4, 128, 256], dtype=torch.bfloat16)
origin_filter = torch.randn([4, 128, 256], dtype=torch.bfloat16)

input_shape = lla.LoweredShape.from_lowered_axis(
    [("h", 1, 4, 32768), ("u", 8, 16, 2048)],
    [("u", 1, 8, 256), ("d", 1, 8, 32), ("c", 1, 32, 1)],
)
filter_shape = lla.LoweredShape.from_lowered_axis(
    [("h", 1, 4, 32768), ("v", 8, 16, 2048)],
    [("v", 1, 8, 256), ("d", 1, 8, 32), ("c", 1, 32, 1)],
)
output_shape = lla.LoweredShape.from_lowered_axis(
    [("h", 1, 4, 32768), ("v", 8, 16, 2048)],
    [("u", 1, 8, 128), ("u", 8, 16, 8), ("v", 1, 8, 1)],
)
```

In progress: PoC-level code

tcp-ir: Low-Level Programming Interface

- DMA
- Fetch Engine
- Contraction Engine

```
tactic = lla.tensor_engine.Tactic(
    slice=[("h", 1, 4, 1), ("v", 8, 16, 1)],
    outer_acc=[("u", 1, 8, 1), ("u", 8, 16, 1)],
    acc=[("d", 1, 8, 1)],
    mac_rows=[("v", 1, 8, 1)],
    lat=[("c", 1, 32, 1)],
)

contraction_args = lla.ContractionArgs(
    input_shape=input_shape,
    input_dtype=torch.bfloat16,
    filter_shape=filter_shape,
    filter_dtype=torch.bfloat16,
    output_shape=output_shape,
    output_dtype=torch.bfloat16,
    tactic=tactic,
)
```

In progress: PoC-level code

tcp-ir: Low-Level Programming Interface

- DMA
- Fetch Engine
- Contraction Engine
- Vector Engine

```
vector_engine = lla.VectorEngine(
    input_shape=fetch_engine.output_shape, input_dtype=float32,
    output_shape=fetch_engine.output_shape, output_dtype=float32,
    # silu(x) = x * sigmoid(x)
    nodes=[
        (lla.vector_engine.Op("split"), []),
        (
            lla.vector_engine.Op(lla.vector_engine.UnaryOp.Sigmoid),
            [lla.vector_engine.OperandSelector()],
        ),
        (
            lla.vector_engine.Op(lla.vector_engine.BinaryOp.MulF),
            [lla.vector_engine.OperandSelector()],
        ),
        (lla.vector_engine.Op("concat"), []),
    ],
)
```

In progress: PoC-level code

Question & Answer

감사합니다.